Unit-II Topic-II Tree

Introduction:

We have studied linked list, which is better data structure as per memory point of view but main disadvantage of linked list is that it is linear data structure. That is every node has information of next node only therefore searching node in linked list is done in sequence manner and hence it is slow. For searching any node in linked list we have to start from first node towards the last node.

Also, some linear data structure i.e. stack, queue and linked list are not suitable to implement directory structure of computer.

In short:

- Searching process is slow in linear data structure
- Linear data structure is not suitable to implement directory structure of computer.

To overcome above drawback of linear data structure, the concept of non-linear data structure (Hierarchical data structure) is introduced. And 'tree' is non-linear data structure.

Tree:

"Tree is non-linear (hierarchical) data structure which is collection of nodes and each node having three parts viz. left, info, right"

Here,

'*left*' is node pointer which holds address of left child

'*info*' is actual data

'*right*' is node pointer which holds address of right child

Also, Tree is dynamic data structure because memory is get allocated for nodes in tree at run time.

Note: Every tree has one external node pointer called 'root' and it always holds address of first node.

Advantages of tree:

- Searching process is fast in tree.
- Tree is suitable to implement directory structure of computer.

Binary Tree:

"Binary tree is collection of finite nodes and it consist of specially designed node called 'root' with maximum two disjoint sub-trees called 'left sub-tree' and 'right sub-tree' "

In binary tree, single node has maximum two branches therefore maximum degree of node in binary tree is *two*.

Also, the node in binary tree having three parts viz. left, info, right

Here,

'*left*' is node pointer which holds address of left child (left sub-tree)

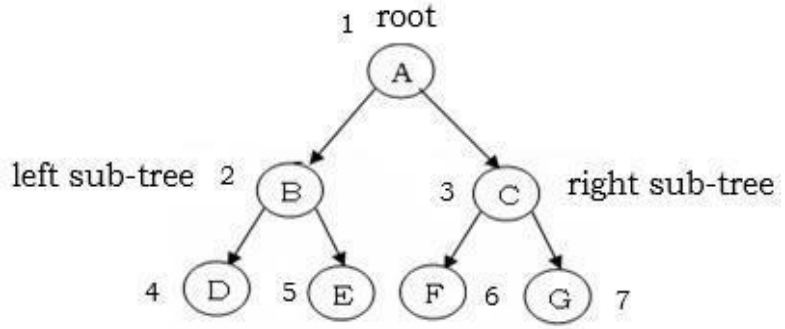
'*info*' is actual data

'*right*' is node pointer which holds address of right child (right sub-tree)

Construction of Binary Tree:

The binary tree can be constructed in top to bottom and left to right manner i.e. Root has number 1, root's left child has number 2, root's right child has number 3 and so on….

Following fig. shows binary tree:



In above fig. 'A' is beginning node of binary tree hence it is 'root' or 'father node'
Node 'B' is left successor of node 'A' therefore it is 'left sub-tree' or 'left child' of father 'A'
Node 'C' is right successor of node 'A' therefore it is 'right sub-tree' or 'right child' of father of 'A'
Similarly, Node 'B' is father of 'D' and 'E'. And node 'C' is father of 'F'

- Terminal Node (External Node or Leaf Node):
  The node which does not have any child is called as "Terminal or leaf or external node"
  In above fig. node 'D', 'E', 'F' and 'G' are leaf nodes.
- Non-Terminal Node (Internal Node or Non-Leaf Node):
  The node that has attached any child is called as "Non-Terminal or Non-Leaf or internal node"

OR

The nodes other than leaf nodes are called "Non-Leaf or Non-Terminal" node.
In above fig. node 'A', 'B' and 'C' are Non-leaf nodes.

- Siblings (Brothers):
  The node which belongs to the same father then they are 'siblings' or 'brothers' of others.
  In above fig. node 'B' and 'C' belongs to same father 'A' therefore they are siblings of each other's.
  Node 'D' and 'E belongs to same father 'B' therefore they are siblings of each other's.

- Degree of node:
  "The number of branches attached to particular node is the degree of that node"
  In above fig. degree of node 'A' is 2
  degree of node 'B' is 2
  degree of node 'C' is 2
  degree of leaf nodes 'D', 'E', 'F' is zero.
- Level of node:
  Root of any binary tree is always considered at level zero. And level of any node is calculated by following formula:

Level of node = Level of its parent + 1

  In above fig. ❋ Level of root 'A' is zero.
  ❋ Level of node 'B' = Level of its parent 'A' +1
  = 0 + 1
  = 1
- Depth (Height) of tree:
  The depth of tree is one more than maximum (largest) level of node in a tree. The depth of tree is calculated by following formula:

Depth of tree =Maximum Level of node + 1 e.g. In above fig. depth of tree = 2 + 1 = 3

- Ancestors:
  The ancestors of any node are all those nodes along within the path from that node to root.
  - Root node does not have any ancestors.

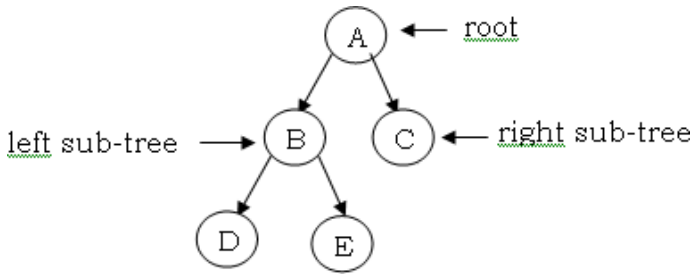
  In above fig. Ancestors of node 'E' are node 'B' and node 'A'
  Ancestors of node 'F' are node 'C' and node 'A'
- Descendants:
  The descendants of any node are all those nodes which are reachable from that node in downward direction.
  - Leaf node does not have descendants.

In above fig. Descendants of node 'B' are node 'D' and node 'E'
Descendants of node 'A' are 'B', 'C', 'D', 'E' and 'F'

Types of Binary Tree:

1) Strictly Binary Tree:

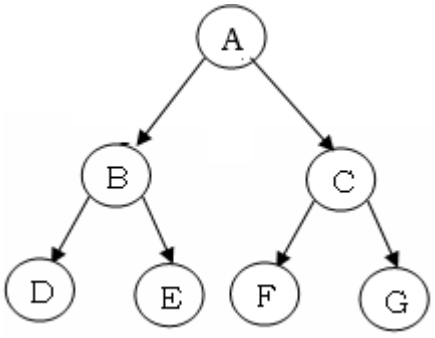
Strictly binary tree is a binary tree in which all non-leaf nodes must have two children or two branches.
Following fig. shows strictly binary tree:



In above fig. Node 'A' and node 'B' are non-leaf nodes and both having two branches that's why above tree is strictly binary tree.

2) Complete Binary Tree: (Full Binary Tree)

Complete binary tree is strictly binary tree in which all leaf nodes are at same level.
Following fig. shows complete binary tree:

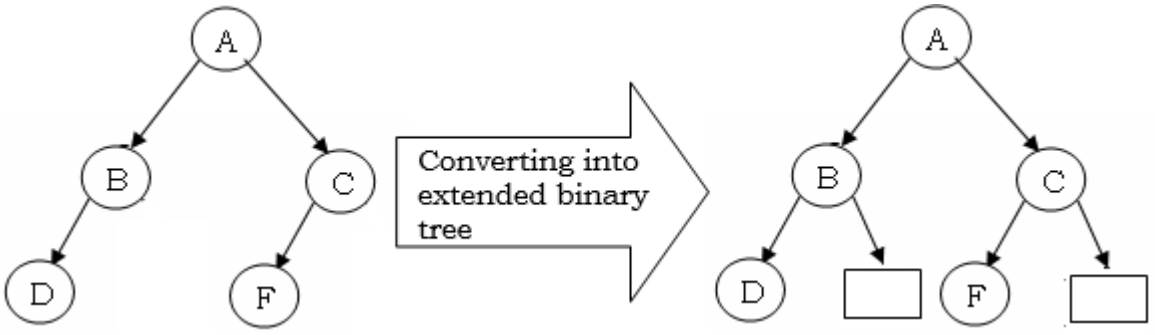


In above fig. It is strictly binary tree and having all leaf nodes i.e. node 'D', 'E', 'F' and 'G' are is at same level therefore it is complete binary tree.

3) Extended (2-Tree) Binary Tree:

A binary tree is said to be 2-tree or extended binary tree if each node in a tree has 0 or 2 children.

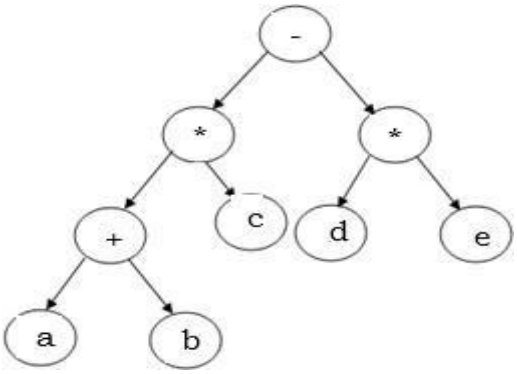
Consider following example;



In above example, we convert binary tree into extended binary tree or 2-tree by adding extra new nodes. The new nodes are external nodes & they are represented by rectangle shape whereas existing nodes are internal nodes & they are represented by circles.

4) Binary Expression Tree:

Any algebraic expression can be represented in binary tree form in such a way that, the leftand right child represents operands whereas the parent node represents the operator.
Let us take an expression: (a+b)*c-(d*e)
Above expression can be representing in binary tree form as follow;

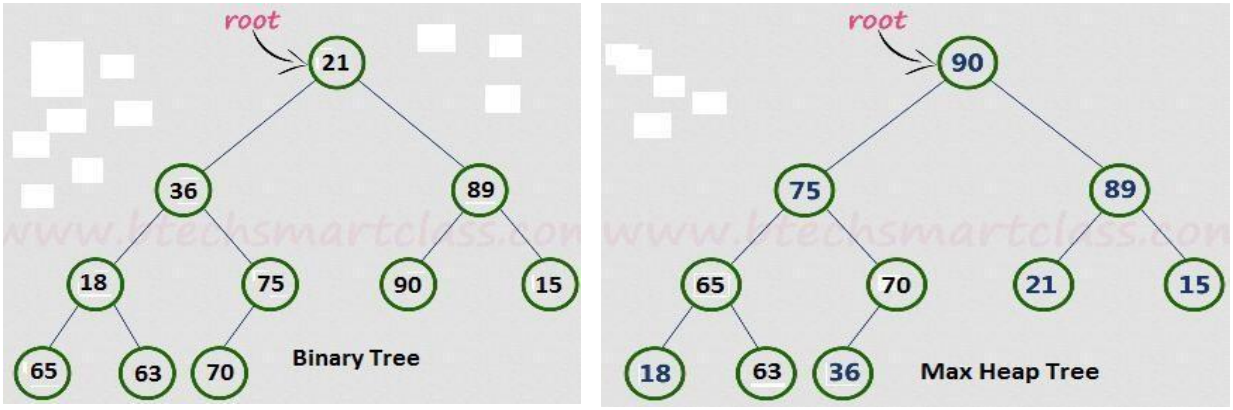


5) Heap Tree:

A Heap tree is one specialized binary tree in which all values are arranged according their values that satisfied heap properties.
Depending on values stored in parent node and child node, heap tree has two types-

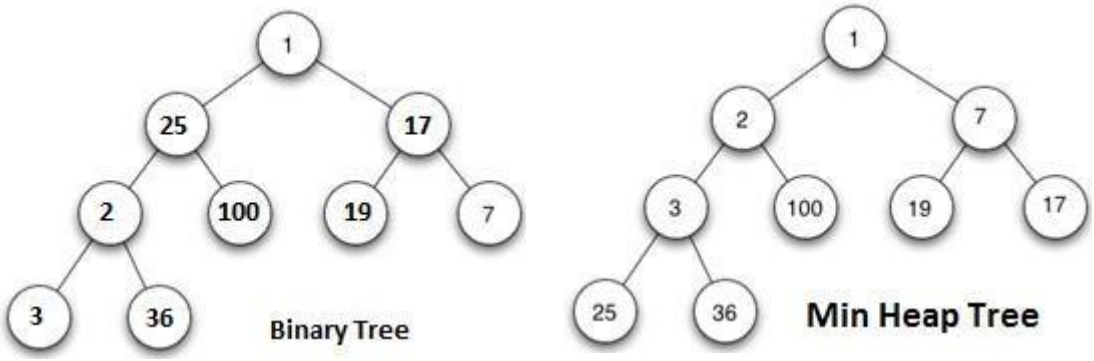
I) Max Heap Tree:

Max Heap tree is type of heap tree in which values (key) in parent node is always greater than or equals to values of its child nodes.
Consider, following binary tree and we can convert it into Max Heap tree as-



II) Min Heap Tree:

Min Heap tree is type of heap tree in which values (key) in parent node is always less than or equals to values of its child nodes.
Consider, following binary tree and we can convert it into Min Heap tree as-

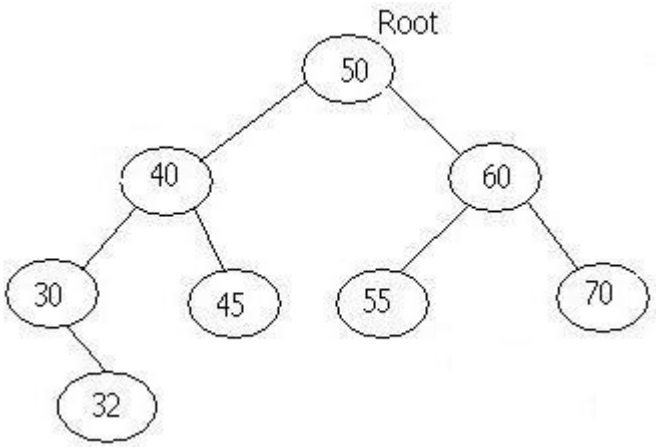


6) Binary Search Tree:

A binary search tree is a binary tree in which all nodes are arranged according to their values. The value of left child is always less than its parent whereas the value of right child is always greater than its parent. That is the nodes belongs to the left side of root are always less than root and the nodes belongs to the right side of root are always greater than root.

e.g. We can make the binary search tree by using following data list:

50, 40, 60, 55, 30, 45, 70, 32

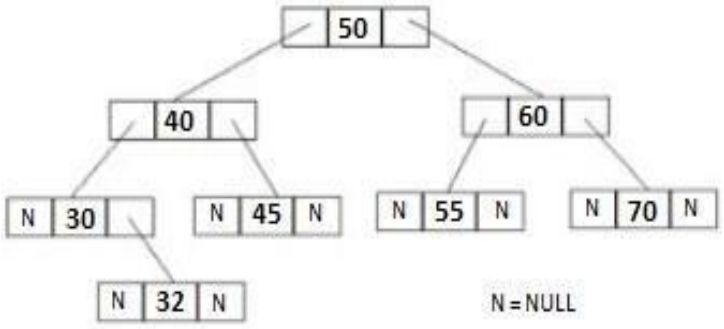


Representation of above binary tree:

We can represent binary tree by two ways-

I) By using Linked list:-

Above binary search tree can be represented by using liked list as follows:



We know that each node in binary tree contains three parts viz.

I) info is used to hold actual data
II) left is node pointer which is used to hold address of left child.
III) right is node pointer which is used to hold address of right child.

Note that: Also, for every binary tree 'root' is external node pointer which always holds address of first node that's why 'root' is header node in tree.

II) By using Array:-

A binary tree can also represents by using array. In array representation, 'root' is stored at index 1, root's left child stored at index 2, root's right child stored at index 3 & so on… Above binary tree can be represent using array is shown in following fig.

Index →	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*
ArrayEle.	50	40	60	30	45	55	70		32

Note:

1) Here, Empty slot is present at index 8, it means node '30' does not have left child.
2) If parent node is at index ***'i'*** then its

Left child's index is= **2i** Right child's index is= **2i+1**

Implementation of binary search tree using linked list:-

For implementation of binary search tree we can define one class which is as follows:

```
class       node
{
        int     info;
        node  *left, *right;
        public:
        // Declaration of Operation of BST.
} *root;
```

Operation performed onto binary search tree:

1) Create operation:

This operation is used to create new node. Also we know that every node in binary search tree are also in dynamic nature.

Operation:

```
node*           create( )
{
        node    *z;
        z = new     node;
        return(z);
}
```

2) Insert operation:

This operation is used to insert new node in binary search tree in such a way that the value of left child is always less than its parent whereas the value of right child is always greater than its parent.

```
void    insert( int     x)
{
        node            *newnode, *temp;;
        newnode = create( );
        newnode→left = NULL;
        newnode→info = x;
        newnode→ right = NULL;
        if ( root = = NULL )        // if tree is empty
        {
                root = newnode;
        }
        else
        {
                temp = root ;           // temp becomes root
                while( temp != NULL )
                {
                        if( newnode→info < temp→info )
                        {
                            if (temp→left = = NULL)
                           {
                                temp→left = newnode;
                                break;
                           }
                           else
                          {
                                temp = temp→left ;
                          }
                        }
                        else    if(newnode→info > temp→info )
                        {
```

```
                    if (temp→right = = NULL)
                    {
                        temp→right= newnode;
                        break;
                    }
                    else
                    {
                        temp = temp→right ;
                    }
                }
                else
                    {
                        cout<< " \n Please do not repeat nodes…";
                        break;
                    }
            }
        }
    }
```

Algorithm (Process) to insert new node into binary search tree: (Creation of binary tree):

Step 1: Start

Step 2: Read any value say 'x'.

Create newnode. And set its three parts as follow:

newnode→left = NULL;

newnode→info = x;

newnode→ right = NULL;

Step 3: Check tree is empty or not as follow:

if (root = = NULL) then // if tree is empty

root=newnode;

goto step 8

Step 4: Assign, temp = root // temp becomes root

Step 5: Check

if(newnode→info < temp→info) then

if *temp's left* is NULL then

Attach newnode to the *left* side of *temp*

goto step 8

else

temp = temp→left // move *temp* to left side

Step 6:

if(newnode→info > temp→info) then

if *temp's right* is NULL then

Attach newnode to the *right* side of *temp*

goto step 8

else

temp = temp→right // move *temp* to right side

Step 7: Repeat step 5 to Step 6, until temp != NULL

Step 8: STOP

3) Search operation:

This operation is used to search particular node is present in binary search tree or not.

Process (Algorithm) to search node:

Step1: Start at the root
Step 2: Compare the search value (key) with the value of the root
Step 3: If the key is equal to root, then display "Node is found" and goto step 7
Step 4: If key is less than the root, then move at the left subtree.
Step 5: If key is greater than the root, then move at the right subtree
Step 6: Repeat steps 2-5 for the root of the subtree chosen in the previous step 4 or 5
Step 7: Stop

Operation:

```
void   search(int    srch)
{
        int     f=0;
        node *temp;
        temp = root;                    // set 'temp' as root
        while( temp != NULL)
        {
                if( srch = = temp→info)  // if search value matches with temp's info part
                {
                        f =1;
                        break;
                }
                else    if (srch < temp→info)     // if search value is less than temp's info
                {
                        temp = temp→left;       // move temp to left side
                }
                else    if (srch > temp→info)     // if search value is large than temp's info
                {
                        temp = temp→right;        // move temp to right side
                }
        }

        if( f = =1)
        {
                Cout<<"\n Node  is  found";
        }
        else
        {
                Cout<<"\n Node is NOT found";
        }
}
```

4) Counting leaf nodes of tree:

This operation is used to count the leaf nodes of binary search tree.

Operation:

```
int     count_leaf(node *p)
{
    if(p==NULL)
    {
      return(0);
    }
    else if(p→left = = NULL&&p→right = = NULL)
    {
        Cout<<"\t"<<p->info;
        return(1);
```

```
    }
    else
    {
        return(count_leaf (p→left) + count_leaf (p→right));
    }
}
```

5) Counting Total nodes of tree:

This operation is used to count the total number of nodes in binary search tree.

Operation:

```
int     count_total (node *p)
{
    if(p==NULL)
    {
      return(0);
    }
    else if(p→left = = NULL&&p→right = = NULL)
    {
      return(1);
    }
    else
    {
        return(count_total (p→left) + count_total (p→right)+1);
    }
}
```

Tree traversal methods:

Traversal: Traversal means visiting all nodes of binary search tree at once, after visiting node we retrieve 'info' part of visited node and display it.

In short, traversing means moving from one node to another node in binary search tree. While doing so, retrieve 'info' part of visited node and display it.

That is, traversal methods are useful to display all nodes of binary search tree.

We can traverse in binary search tree by three ways i.e. there are three traversal methods

1) Preorder traversal (**V**LR) 2) Inorder traversal (L**V**R) 3) Postorder traversal (LR**V**)

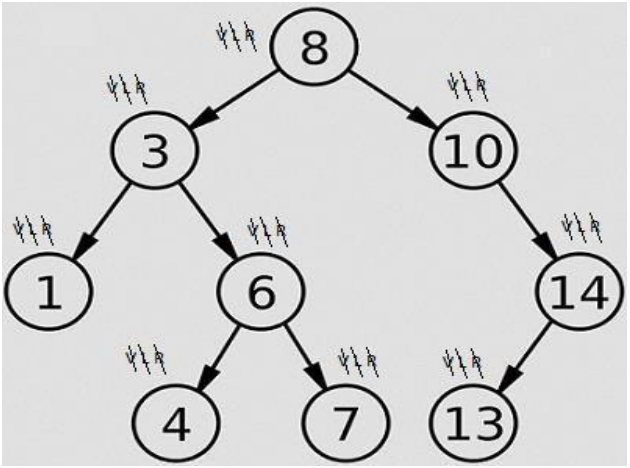
1) Preorder Traversal Method: (VLR method)

Process for preorder traversal method is as follow:

I) Visit the node and retrieve '*info*' part of visited node.
II) Traverse Left sub tree by preorder.
III) Traverse Right sub tree by preorder.

Eg. 1) Create the binary search tree by using following data list and read that tree by using Preorder traversal method.

8, 10, 14, 3, 6, 13, 1, 7, 4



Preorder traversal of above binary search tree = **8, 3, 1, 6, 4, 7, 10, 14, 13**

Operation of preorder traversal:

```
void    preorder(node      *p)
{
        if( p != NULL)
        {
                cout<<"\t"<<p→info;
                preorder(p→left);
                preorder(p→right);
        }
}
```

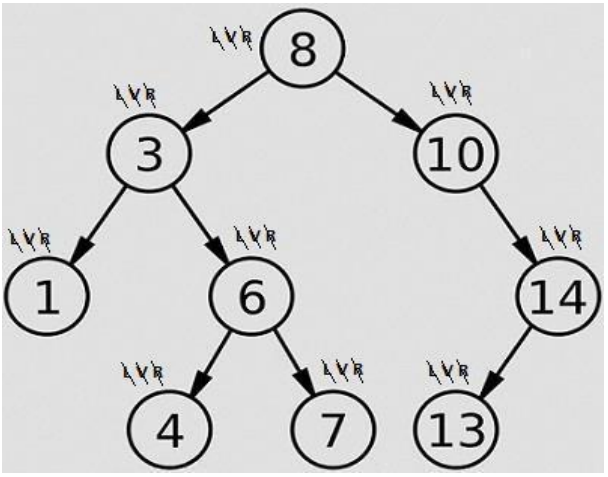
2) Inorder Traversal Method: (LVR method)

Process for inorder traversal method is as follow:

I) Traverse Left sub tree by inorder.
II) Visit the node and retrieve '*info*' part of visited node.
III) Traverse Right sub tree by inorder.

Eg. 1) Create the binary search tree by using following data list and read that tree by using inorder traversal method.

8, 10, 14, 3, 6, 13, 1, 7, 4



Inorder traversal of above binary search tree = **1, 3, 4, 6, 7, 8, 10, 13, 14**

Operation of inorder traversal:

```
void    inorder(node      *p)
{
        if( p != NULL)
        {
                inorder(p→left);
                cout<<"\t"<<p→info;
                inorder(p→right);
        }
}
```

Note: If we read binary search tree by **inorder traversal** method then we get **ascending order** of element.

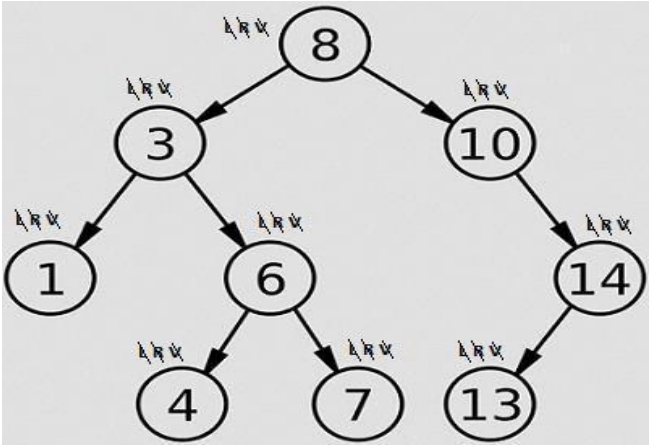
3) Post order Traversal Method: (LRV method)

Process for post order traversal method is as follow:

I) Traverse Left sub tree by post order.
II) Traverse Right sub tree by post order.
III) Visit the node and retrieve '*info*' part of visited node.

Eg. 1) Create the binary search tree by using following data list and read that tree by using post order traversal method.

8, 10, 14, 3, 6, 13, 1, 7, 4



Postorder traversal of above binary search tree = **1, 4, 7, 6, 3, 13, 14, 10, 8**

Operation of postorder traversal:

```
void    postorder(node      *p)
{
        if( p != NULL)
        {
                postorder(p→left);
                postorder(p→right);
                cout<<"\t"<<p→info;
        }
}
```

Deleting (Removing) node from Binary search Tree:

While deleting node from binary search tree we must have to consider following things:

1) First of all, search the node which has to be deleted.
2) After successful search, check type of deleted node i.e. check whether it is leaf, or has attached one child or has attached two children. That is there are three cases to delete the node from binary search tree viz,
   - I) Deleting leaf node
   - II) Deleting node having one child
   - III) Deleting node having two children

Note that:

- While deleting node from binary search tree, we must have to know the location of deleted node and the location of its parent such that proper links to be set out.
- Keep in mind that, after deletion of node, the property of binary search tree (left child value is less than its parent whereas right child value is greater than its parent) can't be violated or disturbed.

Case I) Deleting leaf node:

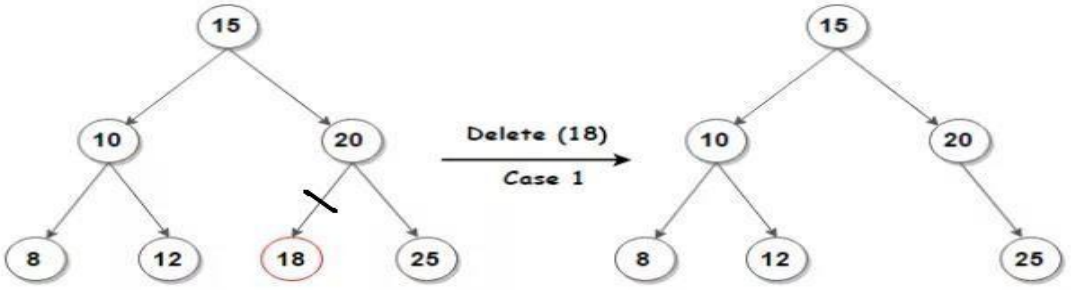
While deleting leaf node from binary search tree, think about it is left child or right child of its parent.

- If leaf node is left child of its parent then set
  parent→left = NULL
- If leaf node is right child of its parent then set
  parent→right = NULL
- And free i.e. delete the leaf node

Operation:

```
void    del_leaf (node *parent, node *child)
{
    if(child == parent->right)
    {
        parent →right=NULL;
    }
    else if(child == parent->left)
    {
        parent →left=NULL;
    }
    delete    child;
    cout<<"\n Node is deleted.....";
}
```

Figure:



Case II) Deleting a node having one child:

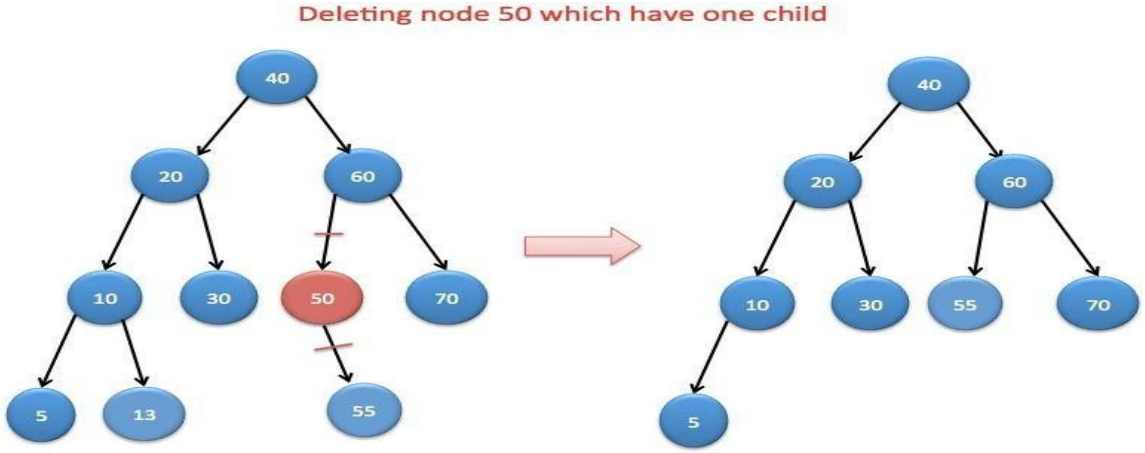
While deleting a node having one child from binary search tree, think about it is left child or right child of its parent.

- If deleted node is left child of its parent then
  attach child's left to parent's left if child's left is not NULL
  else
  attach child's right to parent's left
- If deleted node is right child of its parent then
  attach child's right to parent's right if child's right is not NULL
  else
  attach child's left to parent's right
- And free i.e. delete the node

Operation:

```
void    del_one(node *parent, node *child)
{
   if(child == parent →right)       // if deleted node is parent's right child
   {
            if(child→right ! =NULL)         // if child's right is not null
            {
             parent→right = child →right;   // attach child's right to parent's right
            }
            else       // if child's right is NULL
            {
               parent →right= child →left;    // attach child's left to parent's right
            }
   }
   else if(child == parent →left )    // if deleted node is parent's left child
   {
            if(child→left != NULL)    // if child's left is not null
            {
               parent →left= child →left;    // attach child's left to parent's left
            }
            else       // if child's left in not NULL
            {
               parent →left= child →right;     //attach child's right to parent's left
            }
   }
   delete      child;    // free child
   cout<<"Node is deleted.....";
}
```

Figure:



Case III) Deleting a node having two children:

While deleting a node having two children from binary search tree, do following things.

- **Method-I:** Find right most node (maximum node) from left sub tree of deleted node. And replacevalue of deleted node with right most node (maximum node).
- And delete or free the right most node (maximum node).
  While deleting right most node (maximum node), check it is leaf or having one child.
  I) If right most (maximum node) node is leaf node then call to del_leaf() function.
  II) If right most node (maximum node) has one child then call to del_one() function.

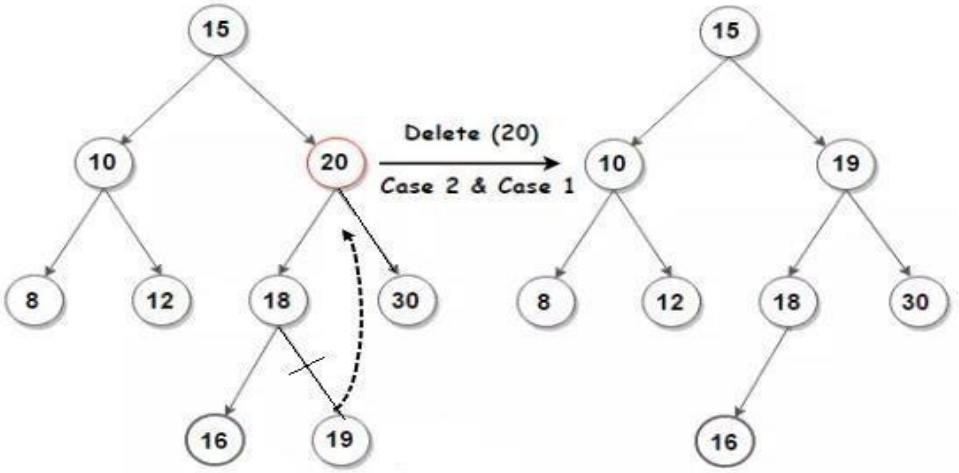
OR

Method-II: Find left most node (minimum node) from right sub tree of deleted node. And replace value ofdeleted node with left most node.

- And delete or free the left most node.
  While deleting left most node (minimum node), check it is leaf or having one child.
  I) If left most node is leaf node then call to del_leaf() function.
  II) If left most node has one child then call to del_one() function.

```
void      del_two(node *child)
{
      node *parent = NULL;
      node *lft=child →left;            // set 'lft' at left of deleted node
      while(lft→right != NULL)      // find right most node (maximum node) from left sub tree of child
      {
           parent=lft;
           lft=lft→right;
      }
    child→info = lft→info;       // replace value of deleted node with right most node( maximum node)
      if(parent == NULL)       // if parent is NULL
      {
           parent = child;
      }
      if (lft→left = = NULL && lft→right==NULL)       // if right most node (maximum node) is leaf
      {
           del_leaf(parent,lft);
      }
      else                                  // if right most node having one child
      {
          del_one(parent,lft);
      }
  }
```

Figure:



- **Balanced Binary Tree Or Height Balanced Tree Or AVL Tree:**
  - We know that, in case of Binary search tree all nodes are arranged according to their values. The value of left child is always less than its parent whereas value of right child is always greater than its parent.
  - But, if we construct binary search tree by taking all nodes in either in ascending or descending order then it causes binary search tree will be skewed (i.e. unbalanced). And retrieving or searching node in such unbalanced tree causes much more time.
  - To overcome this drawback of unbalanced tree, the concept of balanced binary tree is introduced by two Russian mathematicians Adelson Velskii and Landis in year 1962 therefore this tree is also called as AVL tree or Height balanced tree.
  - **Note:** Whenever we add or delete a node of binary tree then balance ness of binary tree disturbed and it will become unbalanced binary tree. And node retrieval or node search process in such unbalanced binary tree becomes slow. To overcome this drawback, the concept of AVL tree is introduced.

Definition: "The binary tree or binary search tree is said to balanced binary tee or AVL tree if the balance factor of every node is in the range **-1, 0, 1**"
Also, AVL tree is extension or specialized binary search tree, where all nodes are inserted and removed by considering balance factor.

The node class for AVL tree is as follow:

```
class      node
{
        int       info, bal_fact ;
        node *left,*right;
        public:
                // operations of AVL tree
} *root;
```

Advantages of AVL tree:
The main advantage of AVL is that, **searching or retrieving** particular node becomes fast because AVL tree is balanced binary tree.

The balance factor of any node is calculated by following formula:

Balance factor (Node) = ht(left child) - ht(right child)

Here, 'ht' means Height
Note that:

- The balance factor of leaf node is always considered as Zero.
- The height of leaf node is always considered as one.
- While performing insert ( ) and delete ( ) operation on to binary tree then balance factor of every node can be changed.
- Right Heavy node: A node is said to be right heavy or right high, if height of its right subtree is more than height of its left subtree.
- Left Heavy node: A node is said to be left heavy or left high, if height of its left subtree is more than height of its right subtree.

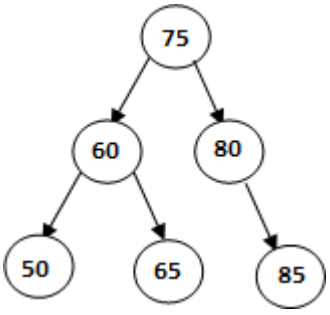
The height of node is calculated by following formula:

1) ht(leaf node) = 1
2) ht(node having one child) = 1 + ht(its child)
3) ht(node having two children) = 1 + max (ht(left child) , ht(right child))

If balance factor of any node is not in the range -1, 0, 1 then that tree is not balanced binary tree or not AVL tree.

Eg : Create binary search tree by using following data list and check that tree is balanced or not.(AVL or not)

75, 80, 85, 60, 65, 50



⇨ We can calculate height of all nodes as follows:

I) Here 85, 65 and 50 are leaf nodes therefore;
ht(85) = 1, ht(65) = 1, ht(50) = 1

II) ht(80) = 1 + ht(85)
= 1 + 1
= 2

III) ht(60) = 1+ max (ht(50) , ht(65))
= 1+ max(1 , 1)
= 1 + 1
= 2

⇨ Now, we can calculate Balance factor of all nodes as follows:

I) Here 85, 65 and 50 are leaf nodes therefore;
bf(85) = 0
bf(65) = 0
bf(50) = 0

II) bf(80) = 0 – ht(85)
= 0 - 1
= -1

III) bf(60) = ht(50) - ht(65)
= 1 - 1
= 0

IV) bf(75) = ht(60) - ht(80)
= 2 - 2
= 0

⇨ In given example Balance factor of every node is in range **-1, 0, 1**

⇨ Therefore given tree is Balanced binary or AVL tree.

Applications of Binary tree:

1) Trees can be used for decision making. Such a tree represents a set of decisions.
2) Binary Search tree is used in many search applications such as the map and set Objects.
3) Binary tree is used in almost every high-bandwidth router for storing router-tables
4) Binary tree is used in almost every 3D video game.
5) Binary trees are used in implementing efficient priority-queues, which are used in scheduling processes in many operating systems.
6) Syntax Trees are Constructed by compilers and (implicitly) calculators to parse expressions.
7) Used in cryptographic applications to generate a tree of pseudo-random numbers.
8) Used to represent directory structure of computer.
9) Binary tree useful for Huffman Code Construction.

Points to remember about Tree:

1) Total number of nodes in complete binary tree having 'n' leaf nodes = $2n-1$
2) Maximum number of leaf nodes in binary tree having height 'h' = $(2)^h$
3) Maximum number of nodes in binary tree having height 'h' = $(2)^{h+1} - 1$ (Since,root is at ht 0)
4) Total number of possible binary trees with 'n' nodes = $(2n)\ !/(n+1)\ !\ *\ n\ !$
5) Maximum number of nodes on level 'x' = $(2)^{x-1}$ (Here, x>=1)
6) If binary tree contains 'n' nodes then $n+1$ links are NULL.

Assignment

Theory

1) What is tree? What are the advantages of tree over linked list?
2) What is Binary tree? List out various types of binary trees.
3) List out applications of tree.
4) What is Binary Search tree? Explain the process to insert new node in binary search tree with its algorithm.
5) Write an algorithm to search the node in BST.
6) What is Binary search tree? Explain its following operations.
   1. Insert 2. Preorder 3. Inorder 4. Postorder
   5. Search 6.Count total nodes 7.Count leaf nodes
   8. Find level of node.
7) What is traversal? Explain different tree traversal methods with its operations.
8) Write short node on: AVL tree (Balanced binary tree)
9) Explain the node delete operation of Binary search tree in details.
10) Explain following terms:

1) Leaf Node or terminal node 2) Non leaf node 3) Siblings or brothers 4) Degree of node 5) Level of node 6) Depth or height of tree 7) Ancestors 8) Descendants	9) Strictly binary tree 10) Complete Binary tree 11) Extended Binary tree 12) Binary Expression tree 13) Max Heap tree 14) Min Heap tree 15) Binary search tree.

Practical Assignment -5

1) Write a program to implement binary search tree with tree traversal methods.
2) Write a program that check entered node is leaf or having one child or having two children.
3) Write a program to implement binary search tree with following operations-
   1) Insert( ) 2) Count_leaf( ) 3) Count_total( ) 4) Search( )
4) Write a program to implement binary search tree with following operations-
   1) Insert( ) 2) find_max( ) 3) find_min( ) 4) display_even( )
   5) display_odd( ) 6) find_level( )
5) Write a menu driven program that deletes node from binary search tree.
   Hint: Menu will look like-

   1: Insert
   2) Inorder
   3) Delete
   4) Exit